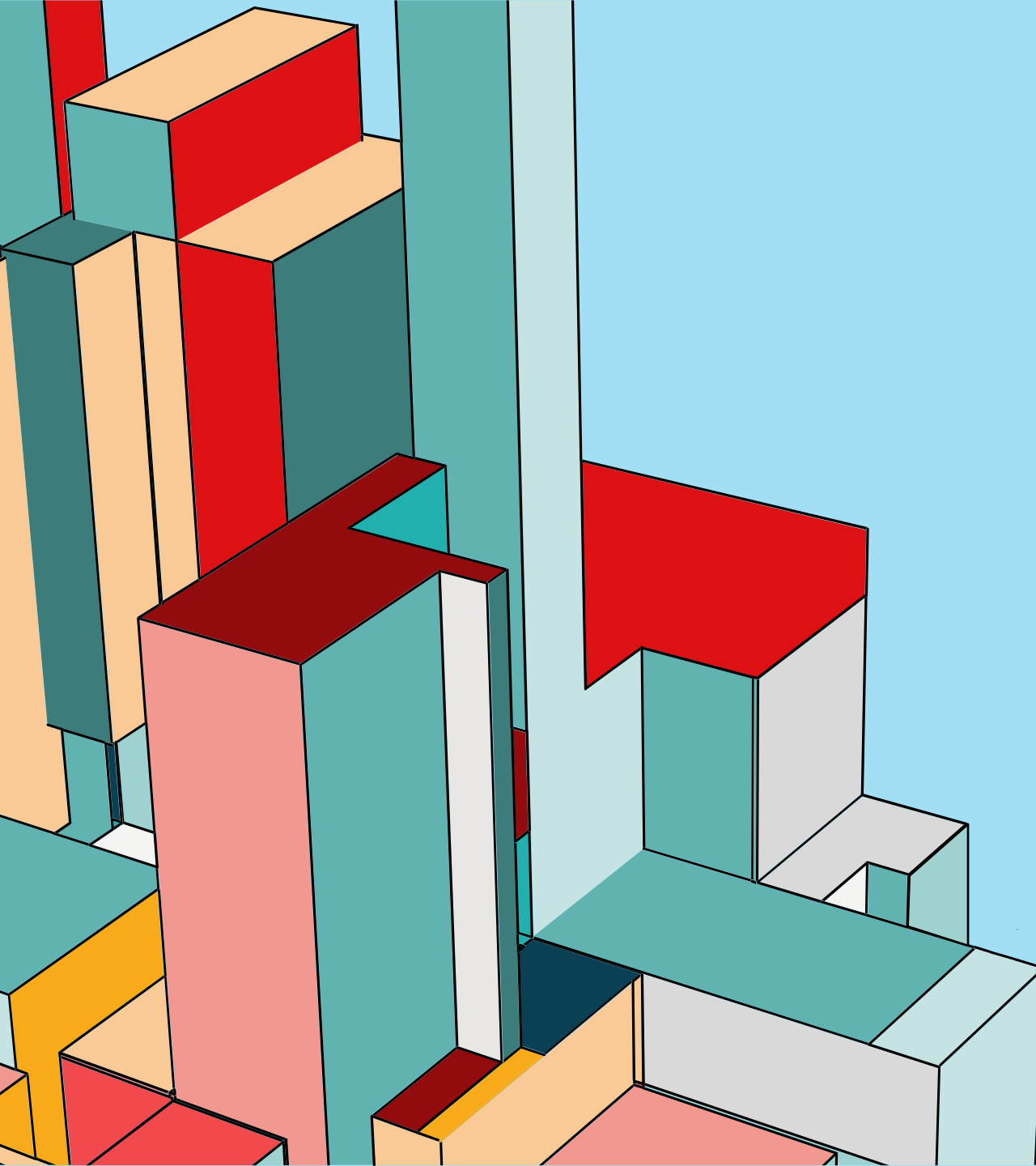
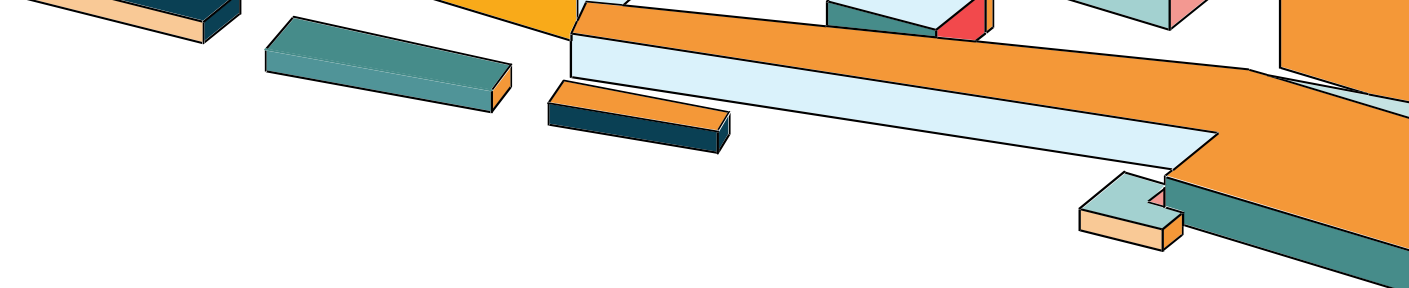




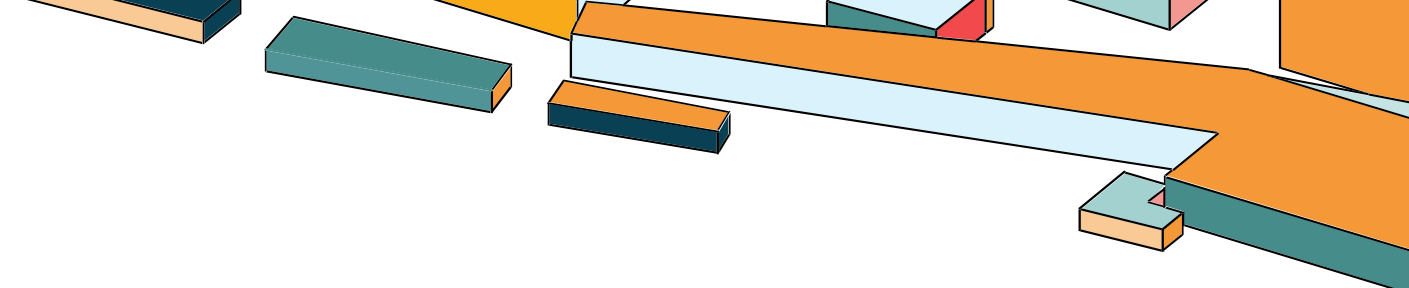
LECTURE 26 **FINAL EXAM REVIEW**

CSIS 2300 Introduction to
Algorithms, Spring 2026
Dr. Dan DiTursi



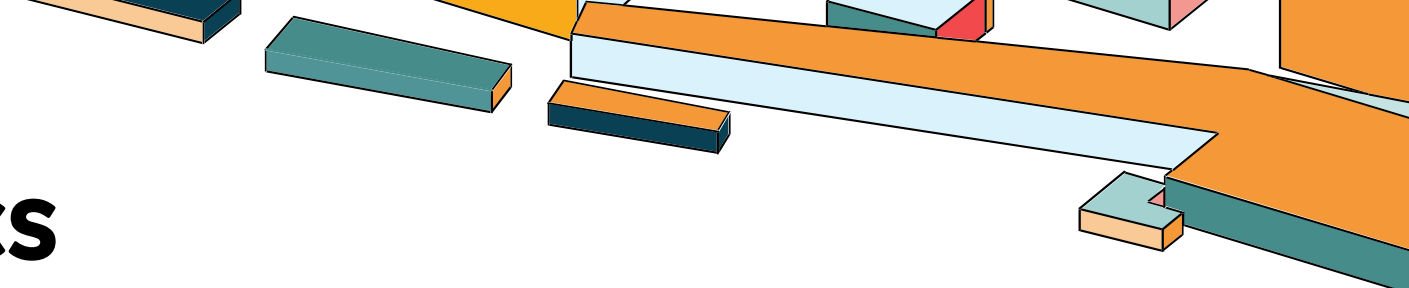
REMINDERS

- HW 11 due **Wednesday** April 29 at 9pm.
 - Goal is to have these returned by Saturday. Inquiries will be due by Sunday!
- Lab sessions this week are optional, with no activity. (i.e. They're extended office hours)
- Course evaluations (assessment.rpi.edu)
 - 166 completed as of 4pm yesterday. 170+ gets bonus on final.
 - Due date Thursday 4/30.
 - Best section to date: 2-4pm (78%); lowest is noon-2pm



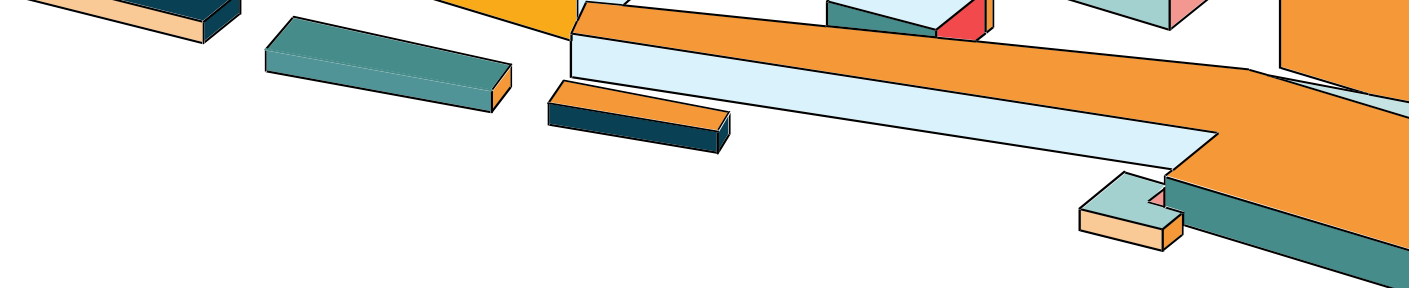
ANNOUNCEMENTS

- Last day for all inquiries except HW 11 will be Saturday May 2.
- DiTursi's office hours this week
 - Tuesday 12:30-14:30 (i.e. the usual)
 - Friday 14:30-16:30 (no Thursday OH this week)
 - TA/mentor office hours should all be on the usual schedule



FINAL EXAM LOGISTICS

- Final exam is Monday May 4, 11:30am-2:30pm
 - Last names A-O: DCC 308
 - Last names P-Z: DCC 318
 - Accommodations: DCC 239 (and see e-mail for times)
- As with previous exams, unlimited paper notes allowed, but they must be either your own or something I have provided.
- You **must** bring your ID with you to the final and present it when submitting your exam.



FINAL EXAM FORMAT

- Aiming for about 1/4 to 1/3 longer than in-semester exams.
- This puts it in the range of 6-9 questions.
 - ~2 execute an algorithm questions
 - 2-3 construct an algorithm questions
 - ~2 analyze an algorithm questions
 - ~2 proofs

SUBMITTY POLL #1

What single topic / type of problem do you LEAST want to see on the final exam?



YOU HAVE TO THINK RECURSIVELY!

The majority of the algorithms we have explored and ALL of the design techniques we have discussed this semester are fundamentally recursive in nature.

The solution to many many problems depends intrinsically upon the solution to one or more smaller instance(s) of the same problem.

This process of analysis – not just breaking down the problem, but seeing how the parts interact and depend upon each other – is really one of the core competencies in the discipline.

It's the difference between a computer scientist and a [code monkey](#).
(Or worse, a prompter.)

KEY ALGORITHMS

- Searches: Linear/binary; BFS & DFS (incl. topological sort)
- Sorts: Iterative (selection, bubble, insertion); recursive (merge, quick*, heap**); non-comparative (radix, counting)
 - * Don't forget Quickselect.
 - ** Includes the linear-time "heapify" algorithm.
- Shortest path: Dijkstra's, Bellman-Ford, Floyd-Warshall
- Spanning trees: Prim's, Kruskal's, deletion ("reverse Kruskal")
- Ford-Fulkerson (finding both max flow & min cut)
- Huffman codes
- Stable matching

ALGORITHM DESIGN TECHNIQUES

- Brute force
- Greedy
 - Must be able to prove optimal or show counterexample
- Decrease-and-conquer
 - Recursive, with ONE subproblem that needs to be solved.
 - Decrease can be by constant amount (e.g. $n - 1$), constant factor (e.g. $n/2$), or vary at each level of the recursion (e.g. quickselect).
- Divide-and-conquer
 - Recursive, with MORE THAN ONE subproblem that needs to be solved.
 - Master Theorem gives us runtime.
- Dynamic programming

OTHER STUFF

- Infrastructure of analysis: Counting critical operations, asymptotic notation (including proofs of these)
- Hashing: open chaining, closed hashing + different types of probing
- P / NP / NP-complete
- Reduction – transforming one problem into another
 - As a technique for building an algorithm
 - As a technique for demonstrating problem difficulty (NP-completeness)

SUBMITTY POLL #2

What single topic / type of problem are you MOST hoping to see on the final exam?



SPECIFIC QUESTIONS?

Anything

FINAL COURSE FEEDBACK FORM

Even if you don't normally fill out the Google poll, please do this one.

